

Build Young — Agent Harness (the loop)

You write a spec → it becomes a task; then the loop drains it one at a time — implement → self-check → verify → risk gate → ship → record → next. Everything is committed in the repo (a fresh container resumes); a portable engineering playbook is the core; always-on machinery guards every step. **Click any section to drill in.**

 human (You)  committed state (files)  AI agent  tooling (local / GitHub)  external (live site)

▼ ① You author the work

— You →  SPECS/ →  TASKS.md · the whole process starts here

- You are the originator **human** — you write a one-page spec, then promote it to a task.
- The spec-first gate: non-trivial work → write a  spec first; a bug-level fix → straight to a task.
- A spec (`SPECS/NNN-name.md`) pins **what / why / behavior / acceptance / out-of-scope / risks** — one page, decisions settled with a human.
- It's promoted into `TASKS.md` as task(s) that reference it. `BACKLOG.md` is the someday parking lot.
- All committed in the repo **state**, so the loop — and a fresh container — resume from it.

▼ ② Kick off a run — three ways the harness runs (they differ in autonomy)

– You — run `/run-loop` locally human

Your Claude subscription; drains the `TASKS.md` backlog. **FULL AUTO** — ships any task the moment the verifier **PASSes**, **regardless of risk**. Spawns a **separate Sonnet sub-agent** as the independent verifier (model tiering — premium doer, cheaper checker).

– GitHub issue + “loop-task” label cloud Action

→ `run-loop.yml` (Anthropic API credits); the issue IS the task. Runs as a **single premium agent** — it can't spawn a sub-agent, so it does its **own** fresh-eyes verification — and **keeps the high-risk pause**: low/med → squash-merge + close the issue; high / architectural / destructive / ambiguous → open a PR, comment WHY, and STOP for a human. (Same spec-first gate as ①: a non-trivial issue is specced first.)

– Scheduled content agent (weekly cron) cloud Action

`content-integrity.yml` — re-verifies external links/stats are primary & current and drafts class materials, then opens **ONE PR for human review**. **Never merges**, never touches `api/` or app behavior — AI judgment where it's safe; the deterministic send/pay code stays untouched.

– Then the SessionStart hook (every session)

Resync to `origin/main` **only if the tree is clean** with no unpushed commits (so it can never clobber work) · reinstall the commit guards into `.git/hooks`. A per-session mechanic, not itself an on-ramp.

Use the local loop OR a labeled issue for a given task, never both. Same loop procedure once started — only the **autonomy** and the **verifier shape** differ.

▼ ③ The loop — pick → self-check → verify → risk gate → ship → record → next

– Main agent — driver + doer agent · premium

Picks the first unchecked task and **reads its** 📄 **spec**, then writes the **smallest change** that meets the acceptance criteria. Never guesses the next step — it comes from a **signal** (a failing test, the verifier's gap report, the next backlog item). Stays in the task's file lane.

– Self-check — gate your own work first

`npm run build · npx vitest run · repo guards` (no internal model-id, no resurrected money-sim markers, diagram currency). Fix until green.

– Verifier — fresh, independent sub-agent agent · sonnet

Own context, inherits nothing — the doer can't grade its own homework. Re-runs build/tests and grades the diff against the acceptance criteria **and the standing rules it is told to read**. Returns **PASS** or **FAIL + gaps**; **fail** → **fix** → **re-verify** (~3 rounds, then stop and surface the blocker). Runs on the cheaper **Sonnet** tier in the **local** loop (cost discipline, rigor unchanged); the **cloud issue Action self-verifies as one premium agent** — it can't spawn a sub-agent.

– Risk gate — reads the task's risk:

Everything is implemented; **only the merge decision differs — and it differs by trigger**: the **local loop is FULL AUTO** (any risk auto-merges once verified — it overrides the playbook's "high-risk pauses" default), while the **cloud issue Action keeps the high-risk pause**. Both stop only on a hard floor (below).

– Ship — on GitHub

`commit` → `push dev branch` → `open PR` → **verify the file diff is non-empty** (container resets have silently dropped commits) → `CI` → **squash-merge** → `main` → `sync`. The same verified path is also a standalone skill — **/ship** — so a **direct edit** merges with identical rigor (build + tests + guards + an independent verifier); verification is a property of shipping, never an unverified shortcut.

– Record — mark [x] in TASKS.md → next task

Progress lives in committed files, so an ephemeral container resumes exactly where it stopped.



The repo — single source of truth

— everything committed, so a fresh container resumes

- `SPECS/` · `TASKS.md` · `.github/workflows` · `src/` (the app) — all versioned together.
- The standing **rules** live here too (next section), so they travel with the code — versioned, reviewable.
- This is what makes the loop resumable: state survives container resets.

▼ **The rules** — the verifier grades every change against these

– **ENGINEERING-PLAYBOOK.md — THE CORE (portable, any repo)**

- **Modularity** — one feature = one file · foundation vs features · single source of truth · thin, data-driven router.
- **Parallel agents** — 4 anti-collision rules: disjoint files · freeze the foundation · contract-first · merge one-at-a-time.
- **Docs as code** — CLAUDE.md is the coordination layer · fix stale docs in the same change · every patterned bug becomes a standing rule.
- **Shipping** — small verifiable increments · never push to main · build + test + guards before every commit · enforce via hooks/CI.
- **Loop engineering (§9)** — goal + acceptance criteria · doer ⇌ verifier split (fresh context) · **model tiering** (premium doer · Sonnet verifier) · risk drives autonomy · spec-first gate is trigger-independent · all state committed.
- **Living architecture (§3)** — the architecture is a committed, auto-regenerated doc with a currency guard (CI blocks a stale one).

– **CLAUDE.md — THIS repo's law (project-specific)**

House style (optimize-for-less-scrolling · no flag/emoji glyphs in data · statistics integrity) · the module/endpoint map · quality bars (WCAG-AA · ~120 KB JS budget · `navLock` · `act()` on clickables). `@imports` the playbook so the doer auto-loads both.

– **POSITIONING.md — voice & claims (project-specific)**

Tagline “raising builders, not consumers” · what we DON'T claim (no income/return promises; build WITH AI, not coding) · the canonical phrasings (e.g. “Talk to us” · the First-year builder prize).

– **How the fresh verifier “knows” to read these**

It inherits none of the doer's context — the spawn prompt **tells** it:  **playbook always** ·  CLAUDE on app/UI ·  POSITIONING on copy ·  the task's SPEC when it references one.

– **The rules self-improve** — `update-playbook` skill

When a session establishes a durable, generalizable lesson (or a rule proves wrong), the agent **proactively** invokes the `update-playbook` skill to record it as **principle + why** in the playbook — so the next change implements the learning instead of re-discovering the bug. Project-specific facts go to CLAUDE.md instead.

▼ 🛡️ **Risk gate & hard floor** — autonomy differs by trigger; both stop only on a hard floor

- **Local /run-loop = FULL AUTO:** any task auto squash-merges the moment the verifier PASSES, **regardless of risk** (the founder opted in — this deliberately overrides the playbook's "high-risk pauses" default). The independent verifier is the gate, not a human.
- **Cloud issue Action = pauses on high-risk:** low/med → auto-merge; **high / architectural / destructive / ambiguous** → **open a PR, comment why, and STOP** for a human to review and merge.
- **Both pause only on a HARD FLOOR:** a genuinely **irreversible real-world side-effect** → confirm first — e.g. **send a real email** · **charge/refund real money** · **delete prod data** · **rotate a live secret**. (Shipping code/copy/refund-math to the repo is reversible via another PR — not a hard floor.)
- A “get a human's eyes eventually” note (e.g. Terms copy) is downgraded to **flag-in-report** in the local loop — surface it in the PR/summary, don't block.

▼ ④ **Fan-out** — default sequential; parallelize only when provably safe

- Go parallel only if **ALL** hold: ≥ 2 tasks on **disjoint files** · **no foundation change** mid-flight · no dependency · the **contract (seams) is pinnable** up front.
- Then: worktree-isolated sub-agents → **merge one-by-one**; after each merge the next rebases.
- **Parallelize the work, not the rigor** — each parallel agent runs the **full loop** (its own verify) on its slice.

▼ ⚙️ **Always-on machinery** — enforced by hooks & GitHub Actions, not memory

- **Commit guards** (installed into `.git/hooks` by SessionStart, so they run however you commit): **hard-block** an internal/bracketed model-id (in the staged diff **and** the commit message) and a **stale diagram** (a hash mismatch if you edit the arch doc/exports without re-rendering); **advisory** on a literal `\uXXXX` escape or a resurrected money-sim marker.
- **CI checks** (path-scoped, block the merge): `architecture-current` (diagram in sync with the source) · `landing-lean` (the marketing page can't re-inflate past its height ceiling). Build + tests are the loop's self-check + the verifier — there's no blanket build/test run on every PR.
- **settings.json** : an allowlist of routine-safe commands (read-only git, build/test) so they don't prompt, + a hard **deny on** `git push ... main` .
- **GitHub MCP** = the agent's API to GitHub (PR · issue · merge). **git worktrees** = fan-out isolation. **SessionStart** = resync + reinstall guards per session.
- GitHub is the hub: the repo (state) · Issues (an on-ramp) · Actions (run-loop · weekly content agent · CI) · Pull Requests (how every change ships).

▼ **Live site — Vercel** — auto-deploys on squash-merge to main

- A **verified** change (any risk) → squash-merge → **deploy**. No manual release step.

▼ **Coordination patterns** — the standard multi-agent catalog, mapped to our components

- **Classify-And-Act** → our risk gate.
- **Fanout-And-Synthesize** → fan-out → merge.
- **Adversarial Verification** → doer ⇌ verifier.
- **Loop Until Done** → the loop itself.
- **Not used (yet):** Generate-And-Filter · Tournament — one doer per task; quality comes from the verifier + guards, not candidate diversity.

This page is the canonical loop diagram (the single source). Static exports are generated from it — `docs/architecture/loop.png` (this overview, collapsed) + `loop.pdf` (the fully-expanded handout) — via `bash scripts/render-architecture.sh`.